

Introduction au langage de commandes (BASH)

Cours n°4
Administration Système – ESIEE – E3INFO
2025/2026

Clément BOIN
clement.boin@esiee.fr

Plan

- Introduction à BASH
- Les caractères génériques
- Les commandes de base
- Les variables
- Les structures de contrôle
- Les tableaux et les fonctions

Introduction à BASH

Bourne-Again Shell

Objectifs d'un langage de scripts

- Automatiser des tâches répétitives
- Automatiser des configurations
- Analyser des journaux de log
- Surveiller le système
 - Régulièrement (planificateur de tâches: cron)
 - En permanence (démons)

Qu'est ce que le shell ?

- Le shell est un programme exécutable, dont la principale fonction est de permettre à l'utilisateur d'interagir avec le système via un terminal.
- Il est aussi appelé *interpréteur de commandes*.
- Deux modes d'utilisation :
 - Interactif : l'utilisateur saisit et exécute ses lignes de commandes une par une dans un terminal ;
 - Non interactif : le shell lit un ensemble de commandes à partir d'un fichier appelé *script*.

Les familles de shell

- Aujourd'hui il existe plus d'une trentaine de shells différents
- Deux grandes familles dominant :
 - Les **cs**h :
 - shells orientés administration, avec une syntaxe inspirée du langage C
 - La famille des **sh** (à l'origine: ash), bsh (Bourne shell), bash (Bourne again shell) :
 - shells orientés utilisateur, majoritaires aujourd'hui.
 - La plupart des scripts shell sont écrits en sh, ou au moins compatibles sh.

BASH

- Le shell UNIX standard est **sh**.
- Mais **Bash** supplante de plus en plus souvent sh (c'est le cas sur Linux) :
 - Il consiste en un mélange de sh, de quelques fonctions du csh, et d'autres du Korn shell (ksh)
 - Il est 100% compatible sh
 - Libre et open source
- Utilisé par défaut sur la distribution **Debian**
 - Permet d'utiliser les **commandes du système**
 - Permet d'écrire des **scripts shell**
 - Vérifie la **validité** des lignes (messages d'erreur)
- Version décembre 2022 : **Bash 5.2**

Un langage de programmation

- Notions présentes:
 - variables (non typées)
 - opérateurs
 - structures de contrôles (if, while...)
 - fonctions...
- Des opérateurs **spécifiques**
 - | « pipe »
 - ;
- Type principal manipulé: **chaînes de caractères**
 - Il existe aussi les entiers, les tableaux...

Avantages

- Langage interprété
- Peu d'erreur de typages
 - chaînes de caractères
- Construction rapide d'application
 - combinaison des commandes existantes
- Possibilité de connecter des composants écrits dans d'autres langages que bash

Inconvénients

- Syntaxe parfois compliquée
- Syntaxe très précise
 - l'oubli d'un espace peut provoquer une erreur
- Caractères ayant des significations différentes selon le contexte
- Plusieurs façons d'écrire la même chose

*

?

[]

~

Caractères génériques

Les caractères génériques: *

- * → désigne n'importe quelle suite de caractères, même vide
- `ls *` affiche tous les noms des éléments du répertoire courant
- `cp a* ~`
 - copie tous les éléments dont le nom commence par a vers le dossier HOME
- `find /etc -name *apache*`
 - recherche tous les éléments qui contiennent la chaîne `apache` dans le dossier `/etc`

Les caractères génériques: ?

- ? → désigne un et un seul caractère
- `ls fichier?` affiche tous les éléments dont le nom commence par `fichier` et suivi d'un seul caractère.
- `ls *fichier?` même chose mais les éléments recherchés peuvent commencer par une chaîne de longueur quelconque.
- **Attention:** si un nom de répertoire correspond au motif, le contenu de celui-ci est affiché par `ls`

Les caractères génériques: []

- [] → désigne un seul caractère parmi une suite de caractères
- `ls fichier[123]` affiche tous les éléments dont le nom commence par fichier et suivi du caractère 1, 2, ou 3
- Notation pour exclure certains caractères:
^
* `[^t o]` désigne les chaînes qui ne terminent ni par t ni par o

Autres caractères : ~ {}

- ~ désigne l'emplacement du dossier de l'utilisateur (contenu de la variable \$HOME)
- ~nom désigne le contenu de la variable \$HOME de l'utilisateur nom
- {} permet de générer des listes de chaînes
 - #copie la séquence de fichiers f1..f5 vers /sauv
`cp f{1..5} ~/sauv/`
 - #Affiche truc1 truc truc8
`echo truc{1,5,8}`

Commandes de base

Exemple d'utilisation

```
$ a=2
```

```
$ echo a
```

```
a
```

```
$ echo $a
```

```
2
```

```
$echo $a+5
```

```
2+5
```

```
$echo $(a+5) #Evaluation arithmétique
```

```
7
```


Commande echo

- **Affiche** les arguments en sortie standard
 - séparations par des espaces

```
$ echo salut tout le monde  
salut tout le monde
```

- Pour conserver la mise en forme, on utilise les guillemets
"

```
$ echo "salut tout le monde"  
salut tout le monde
```

- Avec l'option **-n**, echo ne revient pas à la ligne:

```
$ echo -n bonjour  
bonjour$
```

Typage dynamique

- Il n'y a pas besoin de typer les variables
- Par défaut il s'agit de **chaînes de caractères**
 - Les deux exemples suivants font la même chose

`a=Bonjour`

`a="Bonjour"`

- Une variable est une chaîne même si la déclaration contient des chiffres :

`b=25`

- Il faudra changer de contexte pour faire des **calculs sur des entiers**
- Il n'y a **pas de gestion native des flottants**
- Pas de booléens, juste **0 (vrai!) et 1 (faux!)** dans les codes de retour

Contexte arithmétique

- Pour que Bash réalise un **calcul** avec des nombres il faut utiliser la syntaxe spécifique **`$(( ))`**:

```
x=5
```

```
y=2
```

```
r=$((x+y))
```

- Ou encore la commande intégrée **let**:

```
let r=x+y
```

- Ou même utiliser la commande **expr** (les espaces sont nécessaires):

```
r=$((expr $x + $y))
```

```
r=`expr $x + $y`    #syntaxe alternative
```

Les scripts

Les fichiers shell

- Lorsque plusieurs commandes sont nécessaires, il est possible de les sauvegarder dans un fichier:
 - fichier de commandes, fichier shell, script shell
- La première ligne doit commencer par `#!` (shebang) et préciser l'interpréteur de commande:
 - ici `/bin/bash`

```
#!/bin/bash
# Hello World

echo "Hello World"
```

Les commentaires

- Un commentaire commence par **#** et termine à la fin de la ligne
 - pour être pris en compte il ne doit pas être à l'intérieur d'un mot

```
$ echo bonjour # monsieur
```

```
bonjour
```

```
$ echo # bonjour monsieur
```

```
$ echo do#mineur
```

```
do#mineur
```

Exécution d'un fichier script

- *Première méthode:* la commande bash

```
$ bash script
```

- *Deuxième méthode:* donner les droits d'exécution à l'utilisateur

```
$ chmod u+x script
```

```
$ ./script
```

- Ajout dans le PATH pour éviter de saisir le chemin

```
$ pwd
```

```
/home/moi/bin
```

```
$ echo $PATH
```

```
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
```

```
$ PATH=$PATH:/home/moi/bin
```

```
$ echo $PATH
```

```
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/home/moi/bin
```

```
$ script
```

Des scripts en Python ?

- Il est également possible d'écrire des **scripts** utilisables directement en ligne de commande avec **Python**
 - il faudra rendre le fichier exécutable

Exemple (dans un fichier script.py)

```
#!/usr/bin/env python
while(1):
    print "Bonjour"
```


Exercice 1

- *Ecrire un script qui affiche à l'écran:*

Nous sommes le: mercredi 15 février 2012 15:00:00 (UTC+0100)

- Il faut utiliser la commande `date`



Les variables

Les variables (1/3)

- Une variable est identifié par **un nom**
 - Caractères autorisés: lettres, chiffres, _
 - Sans espace, et ne commençant pas par un chiffre
- Trois **types** de variables:
 - Variables **utilisateurs**: a, b c, somme...
 - Variables **prédéfinies du shell**: PS1, PATH...
 - Variables des **commandes unix**: TERM
- **Affectation** (attention : aucun espace toléré):
`$ nom=valeur`
- **Valeur** d'une variable (caractère \$):
`$ echo nom=$nom`
`nom=valeur`

Les variables (2/3)

La fonction read

- **Saisie** d'une variable:

```
$ read nom prenom
```

```
Aymard Jean
```

```
$ echo $prenom
```

```
Jean
```

- L'option `-p` permet d'afficher **une chaîne d'appel**:

```
$ read -p "Entrez votre nom : " nom
```

```
Entrez votre nom : Jean
```

Les variables (3/3)

La fonction declare

- Permet d'éviter les erreurs de concaténation
- Pour déclarer une **variable en lecture seule**:
`$ declare -r message=salut`
- Pour déclarer une **variable entière**:
`$ declare -i a=16`
- Utile pour déclarer des tableaux (-a) ou des tableaux associatifs (-A)

Les variables spéciales (1/2)

- `$?` → Valeur de retour de la dernière commande, du dernier script...
- `$0` → nom du processus père (le programme)
- `$#` → nombre d'arguments
- `$1, 2, 3... 9` → arguments
- `$*` → tous les arguments (pas `$0`) en une seule chaîne
- `$@` → tous les arguments en autant de chaînes que nécessaire

Les variables spéciales (2/2)

pour les processus

- \$\$ Numéro de processus (PID) du shell courant
- \$! PID de la dernière commande d'arrière-plan terminée
- \$? Code de retour du dernier processus terminé (exit de la dernière commande)

Exercice 2

- *Ecrire les commandes bash nécessaires au calcul et à l'affichage de la somme de 3 valeurs contenues dans des variables nommées r, w et x*
- *Ecrire un script qui prend en paramètres ces trois nombres et affiche la somme*

```
./script 1 3 1
```

```
4
```


Exemple

Affichage des arguments

```
#!/bin/bash
for arg in $*
do
    echo $arg
done
echo "Il s'appelle $0"
echo "Ce programme a $# arguments "
exit 0
```

Une autre version avec l'instruction **shift** :

```
#!/bin/bash
while [ "$#" -gt 0 ] do
    echo "$1"
    shift #Passe à l'argument suivant
done
echo "Il s'appelle $0"
echo "Ce programme a $# arguments"
```

Evaluation des variables (1/2)

- Il est possible de faire remplacer une variable par autre chose que son contenu.
- Le tableau ci-dessous donne un aperçu de quelques substitutions possibles.

Forme	si var définie	si var indéfinie
<code>\${var}</code> ou <code>\$var</code>	Substitue var	Substitue la chaîne vide
<code>\${var:-mot}</code>	Substitue var	Substitue mot
<code>\${var:+mot}</code>	Substitue mot	Substitue la chaîne vide
<code>\${var:?[mot]}</code>	Substitue var	Erreur + substitue mot si spécifié
<code>\${var:=mot}</code>	Substitue var	Affecte mot à var; substitue mot

Evaluation des variables (2/2)

Forme

Substitution

<code>\${#var}</code>	Longueur de la chaîne var
<code>\${var%motif}</code>	var privée de la plus courte occurrence droite de motif
<code>\${var%%motif}</code>	var privée de la plus longue occurrence droite de motif
<code>\${var#motif}</code>	var privée de la plus courte occurrence gauche de motif
<code>\${var##motif}</code>	var privée de la plus longue occurrence gauche de motif

Exemples

<code>\$ var=aaabccc</code>	<code>va: test</code>	<code>\$ echo \${var%e}</code>
<code>\$ echo \$var</code>	<code>\$ echo \${va:?}</code>	<code>aaabccc</code>
<code>aaabccc</code>	<code>va: parameter null or</code>	<code>\$ echo \${var%c}</code>
<code>\$ echo \${va:-mot}</code>	<code>not set</code>	<code>aaabcc</code>
<code>mot</code>	<code>\$ echo \${va:=mot}</code>	<code>\$ echo \${var%%c}</code>
<code>\$ echo \${var:-mot}</code>	<code>mot</code>	<code>aaabcc</code>
<code>aaabccc</code>	<code>\$ echo \$va</code>	<code>\$ echo \${var%c*}</code>
<code>\$ echo \${var:+mot}</code>	<code>mot</code>	<code>aaabcc</code>
<code>mot</code>	<code>\$ echo \${#var}</code>	<code>\$ echo \${var%%c*}</code>
<code>\$ echo \${va:?test}</code>	<code>7</code>	<code>aaab</code>

Les guillemets et les quotes

- La valeur d'une variable (`var=truc`)
 - est **substituée** dans une chaîne entre **guillemets** "
`echo "bonjour $var"`
affiche
`bonjour truc`
 - **n'est pas substituée** dans une chaîne entre **quotes** '
,

```
echo 'bonjour $var'
affiche
bonjour $var
```

Evaluation de commande

- L'évaluation permet à ce que le contenu produit par une commande sur sa sortie standard soit substitué sur la ligne de commande courante.
 - Les délimiteurs autres que l'espace (tabulations, retours chariots) sont remplacés par des espaces.
- **Forme générale:** `$(commande)` ou ``commande``

Exemple

```
$ ls
cap1.tiff          cap2.tiff          test
$ ls | wc -l
3
$ echo Il y a $(ls | wc -l) fichiers
Il y a 3 fichiers
$
```

Les structures de contrôles

La commande if

- Permet de faire des **choix** dans l'exécution:
- Mots clés: **if, then, else, elif, fi**

Exemple:

```
#!/bin/bash
if rm toto > /dev/null
    then
        echo "Toto supprimé "
        exit 0
    else
        echo "Echec"
        exit 1
fi
```

- La fonction rm renvoie 1 (false) si toto a bien été supprimé et 0 sinon (true).
- Le script renvoie 0 si le fichier est bien supprimé, 1 sinon (commande **exit**)

Les expressions conditionnelles (1/4)

Commande composée [[

- Commande **interne** utilisée avec if
- Permet l'**évaluation** d'expressions conditionnelles
 - Autre syntaxe plus portable : [condition]

Syntaxe Bash:

`[[ Condition ]];`

Les espaces
sont obligatoires

Le point virgule
est utile s'il n'y
a pas de saut de
ligne

Exemple

```
#!/bin/bash
if [[ -x test.sh ]]
    then echo "Accessible en exécution"
fi
```


Les expressions conditionnelles (2/4)

Opérateurs logiques

- Parenthèses ()
- Négation !
- & & -> ET logique
- | | -> OU logique

Exemple:

```
#!/bin/bash
if [[ -x test.sh && -a test.sh ]];
then echo "test.sh existe et est accessible en exécution "
elif [[ -s test.sh ]]
then echo "test.sh existe et sa taille est différente de zéro"
fi
```

Les expressions conditionnelles (3/4)

Comparaison de chaînes de caractères:

- Toujours placer les chaînes entre guillemets
- `==` `!=` `>` `<`
- `-z chaîne` -> vraie si la longueur de la chaîne est égale à 0
- `-u chaîne` -> vraie sinon
- Extraction de sous-chaîne : `${ch:2:5}`
 - Affiche "njo" si ch vaut "bonjour"

```
#!/bin/bash
if [[ $1 == "merci" ]]
then echo "de rien"
else echo "malpoli!"
fi
```

Les expressions conditionnelles (4/4)

Comparaison des valeurs numériques

- Commande interne ((
 - Pour effectuer des opérations arithmétiques
+ * /
 - Les opérateurs de comparaison fonctionnent:
== != <= >= < >

Exemple:

```
#!/bin/bash

if (($2 == 2*$1))
    then echo "1er Argument*2 = 2eme argument"
fi
```

Exercice 3

- Ecrire un script qui vérifie si le nombre d'arguments qui lui ont été fournis est égal à deux, et si oui affiche le deuxième argument, sinon un message d'erreur.

```
#!/bin/bash

if [[ $# -eq 2 ]]; then
    echo $2
else
    echo "erreur"
    exit 1
fi
exit 0
```

- Comment vérifier que le premier paramètre est une option et commence par un - ?
Exemple: ./monscript -a 154

```
if [[ $# -ge 1 ]]; then
    arg1=$1
    if [[ ${arg1:0:1} = "-" ]]; then
        echo 'oui'
    else
        echo 'non'
    fi
else
    echo "pas assez d'arguments"
    exit 1
fi
exit 0
```

Les tests possibles

Tests sur les fichiers/répertoires

Voici une liste des tests possibles sur les fichiers et/ou répertoires :

- "-e fichier" : vrai si le fichier/répertoire existe.
- "-s fichier" : vrai si le fichier a une taille supérieure à 0.
- "-r fichier" : vrai si le fichier/répertoire est accessible en lecture.
- "-w fichier" : vrai si le fichier/répertoire est accessible en écriture.
- "-x fichier" : vrai si le fichier est exécutable ou si le répertoire est accessible.
- "-O fichier" : vrai si le fichier/répertoire appartient à l'utilisateur.
- "-G fichier" : vrai si le fichier/répertoire appartient au groupe de l'utilisateur.
- "-b nom" : vrai si nom représente un périphérique (pseudo-fichier) de type bloc (disques et partitions de disques généralement).
- "-c nom" : vrai si nom représente un périphérique (pseudo-fichier) de type caractère (terminaux, modems et port parallèles par exemple).
- "-d nom" : vrai si nom représente un répertoire.
- "-f nom" : vrai si nom représente un fichier.
- "-L nom" : vrai si nom représente un lien symbolique.
- "-p nom" : vrai si nom représente un tube nommé.
- "fichier1 -nt fichier2" : vrai si les deux fichiers existent et si fichier1 est plus récent que fichier2.
- "fichier1 -ot fichier2" : vrai si les deux fichiers existent et si fichier1 est plus ancien que fichier2.
- "fichier1 -ef fichier2" : vrai si les deux fichiers représentent un seul et même fichier.

Tests sur les entiers

- "entier1 -eq entier2" : vrai si entier1 est égal à entier2.
- "entier1 -ge entier2" : vrai si entier1 est supérieur ou égal à entier2.
- "entier1 -gt entier2" : vrai si entier1 est strictement supérieur à entier2.
- "entier1 -le entier2" : vrai si entier1 est inférieur ou égal à entier2.
- "entier1 -lt entier2" : vrai si entier1 est strictement inférieur à entier2.
- "entier1 -ne entier2" : vrai si entier1 est différent de entier2.

Tests sur les chaînes de caractères

Les chaînes doivent être entourées par des guillemets.

- "-n \"chaîne\"" : vrai si la chaîne n'est pas vide.
- "-z \"chaîne\"" : vrai si la chaîne est vide.
- ""chaîne1" = \"chaîne2\"" : vrai si les deux chaînes sont identiques.
- ""chaîne1" != \"chaîne2\"" : vrai si les deux chaînes sont différentes.

Autre syntaxes et commande test

- Afin d'évaluer les différents types de conditions, il existe plusieurs syntaxes en Bash:
 - `[[ condition ]]`
 - `[ condition ];`
- Et la fonction **test** fait la même chose :
 - `test condition`

Exemple dans un script :

```
if test -e $1; then echo "Le fichier existe"; fi
```

Exemple dans le shell (exploitation du code de retour) :

```
$ test -n "$mavar"  
$ echo $?  
1  
$ mavar=4  
$ test -n "$mavar"  
0
```

Code retour 1 (faux) : La variable n'existe pas
Code retour 0 (vrai) : La variable existe

Structure de choix multiple: case

- **Syntaxe:**

```
case valeur in  
  motif)  commande;;  
  motif1|motif2)  commande;;  
  *)  commande;;  
esac
```

- Le shell évalue la *valeur* puis la compare avec les différents modèles (dans l'ordre)
- Dès qu'un modèle correspond, la ou les commandes associées sont exécutées.

- **Exemple:**

```
read -p "Afficher la date?" a  
case $a in  
  o|O ) date;;  
  n*|N* ) ;;  
  * ) echo "erreur";exit 1;;  
esac  
exit 0
```

Les boucles

La commande while

- Même principe que dans les autres langages de programmation:
- **Syntaxe**

```
while conditions  
    do  
    suite_commandes  
done
```

- Exemple:

```
#!/bin/bash  
while who | grep clement > /dev/null; do  
    echo "mystère"  
    sleep 5  
done  
echo "encore plus mystérieux"
```

La commande for

- **Syntaxe:**

```
for var in liste_de_mots  
do  
    suite_de_commandes  
done
```

- La variable var prend **successivement** la valeur de chaque mot de la liste.

- **Exemple:**

```
for i in toto titi tutu  
do  
    echo $i  
done
```

Affiche

```
toto  
titi  
tutu
```

Que fait ce script?

```
#!/bin/bash
```

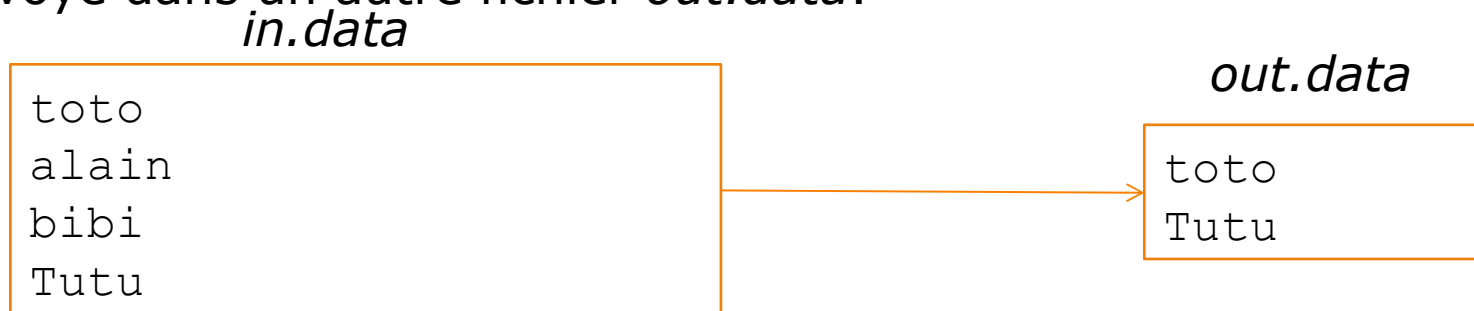
```
for i in $(ls)
do
  if [[ -d $i ]]
  then
    case $i in
    T*) ls $i;;
    *) ;;
    esac
  fi
done
exit 0
```

`$( )` pour stocker le résultat d'une commande dans une variable

`-d` pour vérifier que c'est un dossier

Exercice 4

- Ecrire un script qui prend en argument un fichier *in.data* contenant des noms de personnes et qui filtre ces noms pour ne garder que ceux qui commencent par un t. Les résultat sera envoyé dans un autre fichier *out.data*.



Propositions de correction

```
#!/bin/bash
> out.data

if [[ $# = 1 && -e $1 ]]; then
    for ligne in $(cat $1)
    do
        case $ligne in
            t*|T*) echo $ligne>>out.data;;
            *);;
        esac
    done
else
    echo "erreur" >&2
fi
```

```
#Autre version plus évoluée
if [[ $# -ne 1 || ! -e $1 ]]; then
    echo "erreur" >&2
    exit 1
fi

> out.data

# Lecture ligne par ligne pour éviter les
# problèmes d'espaces
while IFS= read -r ligne; do
    first_char="${ligne:0:1}"
    if [[ "$first_char" == "t" || "$first_char"
    == "T" ]]; then
        echo "$ligne" >> out.data
    fi
done < "$1"

exit 0
```

Les tableaux et les fonctions

Les tableaux indexés

#Déclaration d'un tableau

```
mon_tab=("truc" "bidule" "chose")
```

#Affichage des éléments

```
echo "Éléments du tableau : ${mon_tab[@]}"
```

#Ajout d'un élément

```
mon_tab+=("orange")
```

```
echo "Après ajout : ${mon_tab[@]}"
```

#Accès à un élément

```
echo "Premier élément : ${mon_tab[0]}"
```

#Longueur

```
echo "Taille du tableau : ${#mon_tab[@]}"
```

#Boucle

```
echo "Boucle sur les éléments :"
```

```
for element in "${mon_tab[@]}; do
```

```
    echo ". $element"
```

```
done
```

```
exit 0
```

Les tableaux associatifs

```
# Déclaration d'un tableau associatif
declare -A tab
tab=([prenom]="Toto" [nom]="Bibi" [age]=20)

#Accès aux éléments
echo "Prénom: ${tab[prenom]}"
echo "Nom: ${tab[nom]}"

#Ajout d'un nouvel élément
tab[ville]="Paris"

#Affichage de tous les éléments
for cle in "${!tab[@]}"; do
    echo "$cle: ${tab[$cle]}"
done
```


Les fonctions

Déclaration d'une fonction

```
ma_fonction() { echo "Ceci est ma fonction"; }
```

#Appel de la fonction

```
ma_fonction
```

#Passage de paramètres à une fonction (comme pour un script)

```
addition() {  
    local somme=0 # Variable locale  
    if [ $# -ne 2 ]; then  
        echo "Utilisation : addition nombre1 nombre2"  
        return 1  
    fi  
    somme=$(( $1 + $2 ))  
    echo "La somme de $1 et $2 est: $somme"  
    return 0  
}
```

#Appel de la fonction avec des arguments

```
addition 5 10
```